Titlu: Sistem de autentificare cu încapsulare (CLI)

Cerință generală:
Realizează o aplicație Java (console) care gestionează utilizatori și autentificare, respectând încapsularea: **toate datele sunt `private`** și pot fi modificate/consultate **doar prin metode publice controlate**. **Nu ai voie** să accesezi sau să modifici câmpurile direct din `Main` sau din alte clase.

1) Clasa `Utilizator` (încapsulare strictă)

Creează clasa `Utilizator` cu atribute **private**:

- `username : String`
- `email : String`
- `parolaHash : int` *(NU există getter pentru el)*
- `locked : boolean`
- `failedAttempts : int` *(NU există getter pentru el)*

Constructor obligatoriu:

- `Utilizator(String username, String email)`
  Reguli:
- `username` nu e gol și are minim 3 caractere (altfel aruncă `IllegalArgumentException`)
- `email` conține `@` și `.` (altfel aruncă `IllegalArgumentException`)
- la început: `locked=false`, `failedAttempts=0`, `parolaHash=0` (parola „nesetată”)

Metode obligatorii:

- `String getUsername()`
- `String getEmail()`
- `boolean isLocked()`
- `boolean setParola(String parola)` cu reguli:
  - minim 8 caractere
  - conține cel puțin 1 cifră
  - conține cel puțin 1 literă mare
    Dacă parola e validă, setează `parolaHash` (folosind o funcție internă de hash) și returnează `true`, altfel `false`.
- `boolean verificaParola(String parola)`
  Returnează `true` dacă hash-ul parolei introduse coincide cu `parolaHash` (fără să expui hash-ul).
- `boolean autentifica(String parola)`
  Reguli:
  - dacă `locked == true` → return `false`
  - dacă parola e corectă → `failedAttempts = 0`, return `true`
  - dacă parola e greșită → `failedAttempts++`; la `failedAttempts >= 3` setează `locked = true`; return `false`
- `toString()` afișează **doar**: username, email, locked (NU hash, NU contor)

Observație: `calcHash(String s)` trebuie să fie metodă internă (ex: `private int calcHash(...)`).

2) Clasa `AuthService` (încapsulare pe colecție)

Creează clasa `AuthService` care gestionează utilizatorii în memorie.
Atribut **private**:

- `users : ArrayList<Utilizator>` *(NU returna lista direct)*

Metode obligatorii:

- `boolean register(Utilizator u)`
  Reguli: respinge `null`; respinge username duplicat; dacă e OK, adaugă și returnează `true`
- `Utilizator findByUsername(String username)`
  Caută după username folosind `.equals()`

- `boolean login(String username, String parola)`
  Găsește utilizatorul și apelează `u.autentifica(parola)`; dacă nu există → `false`
- `String listUsers()`
  Returnează un text cu utilizatorii (folosind `toString()` din `Utilizator`)

3) Clasa `Main` (CLI cu meniu)

Implementează un meniu în `Main` (cu `Scanner`) cu opțiunile:

- `1) register` → cere `username`, `email`, creează utilizator și îl înregistrează
- `2) set password` → cere `username`, `parola`, setează parola (dacă user există)
- `3) login` → cere `username`, `parola`, afișează `LOGIN OK` / `LOGIN FAIL` și dacă user e blocat afișează `LOCKED`
- `4) list users` → afișează lista utilizatorilor
- `0) exit`

Reguli obligatorii în Main:

- Nu folosi câmpuri direct (nu există acces la ele oricum)
- Verifică existența utilizatorului (dacă `null` → mesaj clar)
- Citește opțiunile cu `nextLine()` și parsează ca să eviți problemele cu newline

Testare obligatorie (minim 3 scenarii)

1. Parolă invalidă (ex: `abc`) → trebuie respinsă
2. Parolă validă (ex: `Abcdefg1`) → acceptată
3. 3 autentificări greșite consecutive → utilizatorul devine `LOCKED` și nu mai poate intra

Livrabile

- `Utilizator.java`
- `AuthService.java`
- `Main.java`
  Aplicația trebuie să ruleze din consolă și să respecte toate regulile de încapsulare.

Pentru acasă realizați interfață acestei aplicații.